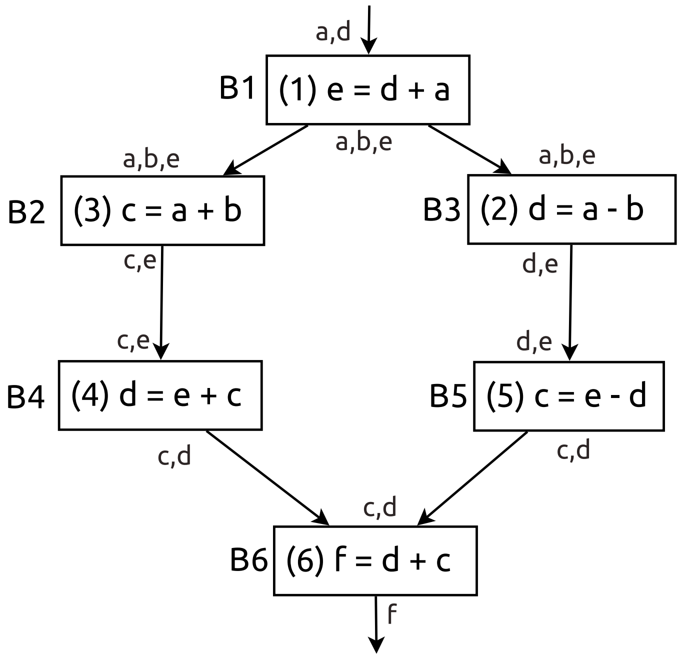
Register Allocation & Garbage Collection
Compilers

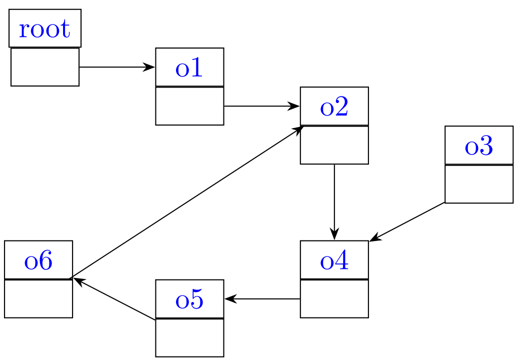
---

1. Consider the following flow graph:



   Perform register allocation using Chaitin's graph-coloring register allocation algorithm, assuming that you have 3 registers.

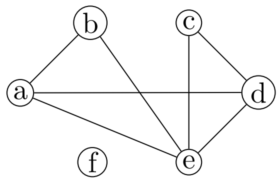
2. Consider the following graph:



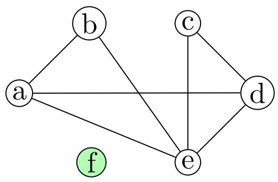
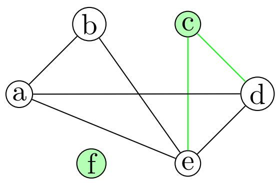
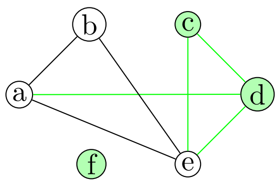
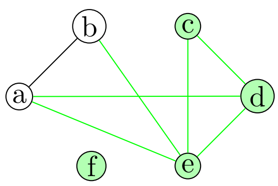
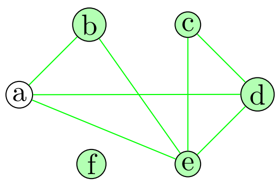
   We indicate the names of the objects within the objects themselves. Execute Baker's garbage collection algorithm, indicating how the `free`, `Unreached`, `Unscanned`, and `Scanned` sets evolve. We assume that initially `free` $= \emptyset$ and `Unreached` $= \{\text{root}, o1, o2, o3, o4, o5, o6\}$, i.e., we are using the entire memory.

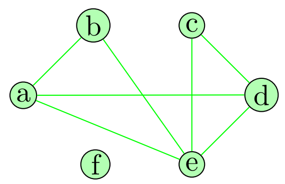
Solution

1. We first create the corresponding graph:

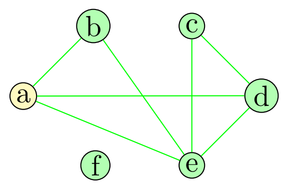
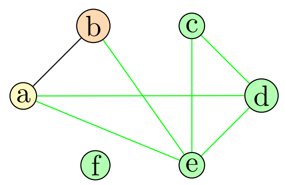
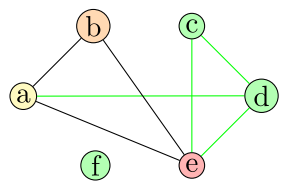
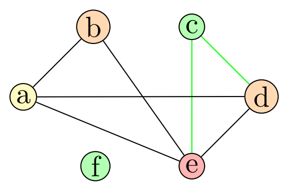
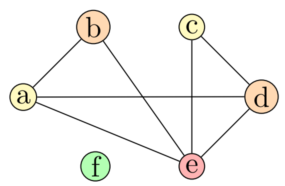
   

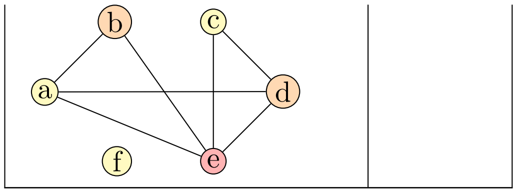
   We then start stacking our nodes:

Graph	Stack
	f
	c f
	d c f
	e d c f
	b e d c f

<table>
<tr><td>
</td><td>a
b
e
d
c
f</td></tr>
</table>

We then color our graph:

<table>
<tr><th>Graph</th><th>Stack</th></tr>
<tr><td>
</td><td>b
e
d
c
f</td></tr>
<tr><td>
</td><td>e
d
c
f</td></tr>
<tr><td>
</td><td>d
c
f</td></tr>
<tr><td>
</td><td>c
f</td></tr>
<tr><td>
</td><td>f</td></tr>
<tr><td></td><td></td></tr>
</table>



This results in a coloring that does not require spilling any variable.

2. Here is an execution of Baker's algorithm:

   (a)
   - `free` $= \emptyset$
   - `Unreached` $= \{\text{root}, o1, o2, o3, o4, o5, 06\}$
   - `Unscanned`
   - `Scanned`

   (b)
   - `free` $= \emptyset$
   - `Unreached` $= \{o1, o2, o3, o4, o5, o6\}$
   - `Unscanned` $= \{\text{root}\}$
   - `Scanned` $= \emptyset$

   (c)
   - `free` $= \emptyset$
   - `Unreached` $= \{o2, o3, o4, o5, o6\}$
   - `Unscanned` $= \{o1\}$
   - `Scanned` $= \{\text{root}\}$

   (d)
   - `free` $= \emptyset$
   - `Unreached` $= \{o3, o4, o5, o6\}$
   - `Unscanned` $= \{o2\}$
   - `Scanned` $= \{\text{root}, o1\}$

   (e)
   - `free` $= \emptyset$
   - `Unreached` $= \{o3, o5, o6\}$
   - `Unscanned` $= \{o4\}$
   - `Scanned` $= \{\text{root}, o1, o2\}$

   (f)
   - `free` $= \emptyset$
   - `Unreached` $= \{o3, o6\}$
   - `Unscanned` $= \{o5\}$
   - `Scanned` $= \{\text{root}, o1, o2, o4\}$

   (g)
   - `free` $= \emptyset$
   - `Unreached` $= \{o3\}$
   - `Unscanned` $= \{o6\}$
   - `Scanned` $= \{\text{root}, o1, o2, o4, o5\}$

   (h) Since $o6$ points to $o2 \notin$ `Unreached`, then we do not need to deal with $o2$ again:
   - `free` $= \emptyset$
   - `Unreached` $= \{o3\}$
   - `Unscanned` $= \{\}$
   - `Scanned` $= \{\text{root}, o1, o2, o4, o5, o6\}$

(i) We stop since `Unscanned` $= \emptyset$, and finally update `free` and `Unreached`:

- `free` $= \{o3\}$
- `Unreached` $= \{\text{root}, o1, o2, o4, o5, o6\}$
- `Unscanned`
- `Scanned`

We see here that we have now managed to reclaim $o3$.